

Your question about pulling the two angled sides exposes a real problem: you should not pull the hexagon at all for material verification.

Pulling a hexagon with angled faces does **not** create a uniaxial stress state:

- The angled faces generate **off-axis reaction components**.
- The strain field is **non-uniform** (the shape itself causes gradients, independent of the holes).
- The reaction force you read is therefore **not cleanly attributable** to E_2 or E_3 — you would be comparing two contaminated numbers and hoping the contamination cancels.

Instead, verify the material card on a RECTANGULAR BLOCK cut from the tube field — same hole pattern, same thickness, flat parallel faces. Clean uniaxial stress state, reaction force that actually means something. The hexagon outline is irrelevant to *material* verification.

Then, separately, run **§B.5 structural correlation** on the real hexagon — load the actual part in both models and compare displacements. That validates the card in the true geometry without pretending it is a material test.

B.1 Build Model D (detailed, rectangular block)

1. **Part** → 3D, Deformable, Solid, Extrusion.
2. **Sketch a rectangle** cut from the tube field — make it an **integer number of lattice cells** so the hole pattern is complete at the edges. Something like **~5 × 5 cells**: $\boxed{L_x \approx 5 \times 0.505 = 2.525"}$, $\boxed{L_y \approx 5 \times 0.50203 = 2.510"}$. Bigger is better (less edge influence); ~5+ cells per side is a reasonable minimum.
3. **Extrude depth** = **2.985"** (Z).
4. **Cut the hole array**: Ø0.32 circles at the real lattice positions ($\boxed{a_1 = (0.505, 0)}$, $\boxed{a_2 = (0.25194, 0.50203)}$), Through All.
5. **Material: ISOTROPIC** ($\boxed{E_m}$, $\boxed{\nu_m}$). Section → Solid, Homogeneous → Assign.
6. **Mesh**: C3D20R, hex sweep, ≥ 3 quadratic elements across the 0.185" min ligament (see §12).

B.2 Build Model E (equivalent, same block, no holes)

1. **Same rectangle, same thickness — NO holes**. A plain solid block, identical outer dimensions to Model D.
2. **Material** → Create → Mechanical → Elasticity → Elastic → Type: **ENGINEERING CONSTANTS**. Enter the nine values in Abaqus's order:

E1, E2, E3, Nu12, Nu13, Nu23, G12, G13, G23

Using the card (§13.0) with **material 1 = Z**:

```
E1 = 0.683*E_m      ← through-thickness (Z)
E2 = 0.34*E_m      ← in-plane
E3 = 0.34*E_m      ← in-plane
Nu12 = 0.30
Nu13 = 0.30
Nu23 = 0.34
G12 = R_Gx*G_m
G13 = R_Gx*G_m
G23 = 0.127*E_m
```

3. ***ORIENTATION** — **mandatory**. Property → **Material Orientation** → Assign. Define a **rectangular datum CSYS** whose **local axis 1 points along global Z**. (Easiest: create a datum CSYS with axis-1 = global Z, axis-2 = global X, axis-3 = global Y, then assign it.)
 - **Get this wrong and E_1/E_2 swap** — a silent, very common failure. See the sanity check in B.4.
4. **Mesh**: C3D20R. Coarse is fine (no holes). But **converge it independently** — do not let a mesh difference between D and E masquerade as a material-card error.

B.3 Step, BCs, and loading — prescribed displacement

Step → Create → Static, General. NLGEOM off.

Apply the SAME BCs to both models. Use **prescribed displacement**, read **reaction force**.

Why displacement, not pressure/traction: a perforated plate under uniaxial *load* develops a **non-uniform strain field** — there is no clean average strain to compare against the homogeneous model. Displacement control gives a well-defined reaction resultant on both. This is the key methodological point.

Case 1 — Z pull (verifies $E_1 = 0.683 \cdot E_m$)

- **BC-1 (fix):** bottom face ($Z = 0$): $U_3 = 0$.
- **BC-2 (RBM restraint, minimal):** on the bottom face, pin **one node**: $U_1 = U_2 = 0$. On a second node along the X-axis: $U_2 = 0$. (*That is it — do NOT restrain the whole bottom face in-plane.*)
- **BC-3 (load):** top face ($Z = 2.985$): $U_3 = \delta$, e.g. $\delta = 0.002985$ (= 0.1% strain).
- Leave all **side faces free** (traction-free) so Poisson contraction is unimpeded.

Case 2 — Y pull (verifies E_2 , weak direction)

- **BC-1:** face at $Y = 0$: $U_2 = 0$.
- **BC-2 (minimal RBM):** one node on that face: $U_1 = U_3 = 0$; a second node: $U_3 = 0$.
- **BC-3:** face at $Y = L_y$: $U_2 = \delta_y$ (e.g. 0.1% of L_y).
- All other faces **free**.

Case 3 — X pull (verifies E_3 , strong direction)

- Same pattern, swapping X for Y.

Optional Case 4 — in-plane shear (exercises G_{23})

- Fix the $Y = 0$ face fully; on the $Y = L_y$ face prescribe $U_1 = \delta$ (tangential), $U_2 = 0$.

The single biggest BC error: restraining the whole loaded/fixed face in *all* DOF. That injects Poisson stiffening, makes **both** models read stiff, and can **mask a bad card**.
Restrain only what is needed to kill rigid-body motion.

B.4 Reading the reaction force and comparing

B.4.1 How to read RF

1. **Group the loaded face into a node set** (Assembly → Set → Node) — call it **SET_LOAD**.
2. **Field output:** request RF and U.
3. **In Visualization:**
 - **Report → Field Output** → Position: **Unique Nodal** → Variable: **RF, RF3** (for a Z pull) → select **SET_LOAD** → check **Sum** in the setup → Write.
 - The **sum of RF3 over the loaded face** = the total reaction force **R**.
 - (Alternative: *History Output on the set, or Tools → Query → Probe values with sum.*)
4. **Structural stiffness:** $K = R / \delta$.

B.4.2 Compare

For each case, compute K_D (detailed) and K_E (equivalent):

$$\text{error} = (K_E - K_D) / K_D$$

You can also back out an apparent modulus ($E_{\text{app}} = K \cdot L / A_{\text{gross}}$) for an intuitive cross-check against $0.683 \cdot E_m$ and $0.34 \cdot E_m$ — using the **gross** area, since that is what the equivalent model assumes.

B.4.3 Acceptance criteria

Case	Expect	Interpretation
1 — Z pull	within 2-3%	E_t is a near-exact closed form. A bigger gap = mesh or BC bug .
2 — Y pull (weak)	within ~10%	The card was built at this direction's η , so it should match here best.
3 — X pull (strong)	Model E noticeably too soft	Expected and correct. You collapsed a directional array to one isotropic $\eta = 0.366$ (the weak one), so E must under-predict the strong direction. This asymmetry is the worst-case conservatism showing up exactly where it should.

Red flag: if Model E is **stiffer** than Model D in-plane, the card is **non-conservative** → wrong η , chart misread, or a bad orientation.

Orientation sanity check (catches the most common error): Model E's **Z stiffness should be roughly 2× its in-plane stiffness** (0.683 vs 0.34). If it is not, your *ORIENTATION axes are swapped.

B.5 Structural correlation on the real hexagon (separate check)

Once the card passes on the block, confirm it in the **actual geometry**:

1. Build the **real hexagon** twice — Model D-hex (holes meshed, isotropic) and Model E-hex (no holes, orthotropic card in the tube field, isotropic base metal in the untubed border, partitioned $\sim \boxed{D_h/4 = 0.08''}$ beyond the outer hole centerlines per §10.6).
2. Apply a **representative service load** — the loading the tubesheet actually sees (e.g. pressure on a face), **not** an artificial pull on the angled sides.
3. **Compare global displacements** at several probe points, and total strain energy.
4. **Accept** if displacements agree within ~10% and strain energy within ~10%.

This validates the card under the real boundary conditions and shape **without** requiring a clean uniaxial state — which the hexagon cannot provide.

13.4 MATLAB — assemble C* and read the constants

matlab

14.11 GUI (Abaqus CAE) Walkthrough

Honest caveat up front: CAE's substructure support is **partial**. Generation and usage exist under **Model → Substructure**, but two things have **no GUI equivalent** and must be added via the **Keyword Editor**:

- `*ELEMENT RECOVERY MATRIX` (scoping recovery to the hole elements)
- `*SUBSTRUCTURE MATRIX OUTPUT` (writing the readable `.mtx`)

Both are flagged [**KEYWORD EDITOR**] below. Everything else is clickable.

14.11.0 Finding the centroid of the hexagonal tubesheet

You need this for `RP_BOT` / `RP_TOP`. **Do not eyeball it.**

Method 1 — query the mass properties (best, exact):

1. Switch to the **Part** module, with the tubesheet part displayed.
2. **Tools → Query...**
3. Select **Mass properties** (under General Queries).
4. Pick the solid part → **Done**.
5. The message area prints **Volume**, **Mass**, and **Centroid (X, Y, Z)**. Read the X, Y directly.

This is the true area/volume centroid of the *actual* hexagon, including the angled sides — which is exactly what you want and what no hand sketch will give you reliably.

Method 2 — probe the face centroid:

1. **Tools → Query → Probe values**.
2. Set **Probe: Elements** (or Nodes), pick the face of interest.
3. Less direct than Method 1; use it as a spot check.

Sanity check by hand: your hexagon has a flat bottom, two vertical sides, two inward-angled sides, and a flat top — so it is **symmetric about its vertical centerline**. Therefore **X_centroid = the midpoint of the flat bottom edge** (by symmetry). Only **Y_centroid** requires the query. If Method 1's X differs from your symmetry axis, something is off in the geometry.

Using it: the two reference points share the same (X, Y) = the centroid, and differ only in Z:

```
RP_BOT = (X_c, Y_c, 0.0)
RP_TOP = (X_c, Y_c, 2.985)
```

Note: the centroid is the natural, unbiased choice for the reference point, but with **kinematic coupling** the RP location does **not** change the physics — the face is rigid, so any RP on it gives an equivalent (just differently-referenced) stiffness. The centroid keeps the moment terms clean and interpretable, which is why we use it.

JOB 1 (GUI) — Generate the substructure

Step 1 — Property module: isotropic material

1. **Module: Property**
2. **Material** → **Create**. Name: `INCONEL_650F`.
3. **Mechanical** → **Elasticity** → **Elastic**. Type: **Isotropic**.
4. Young's Modulus = `27e6`, Poisson's Ratio = `0.3`. **OK**.
 - (§14.0: isotropic — **not** the §9 orthotropic card, because the holes are meshed.)
5. **Section** → **Create**. Solid → Homogeneous. Material: `INCONEL_650F`. **OK**.
6. **Assign** → **Section**. Select the whole part → **Done** → **OK**.

Step 2 — Assembly module: instance + reference points

1. **Module: Assembly** → **Instance** → **Create** → select the part → **Independent (mesh on instance)** → **OK**.
2. Get the centroid (§14.11.0).
3. **Tools** → **Reference Point**. Enter coordinates `(X_c, Y_c, 0.0)` → this is **RP-1** (`RP_BOT`).
4. **Tools** → **Reference Point** again: `(X_c, Y_c, 2.985)` → **RP-2** (`RP_TOP`).
5. **Tools** → **Set** → **Create** → Type: **Geometry** → name `RP_BOT_SET`, select RP-1 → **Done**. Repeat for `RP_TOP_SET`.

Step 3 — Define the interface surfaces

1. **Tools** → **Surface** → **Create**. Name: `SURF_BOT`. Type: **Geometry**.
2. Select the **bottom face** ($Z = 0$) of the perforated region → **Done**.
3. Repeat for `SURF_TOP` (the $Z = 2.985$ face).
4. **Tools** → **Set** → **Create** → Type: **Element** → name `ELS_HOLES` → select the elements **ringing the holes** (the ones whose stress you want to recover).
 - *Tip:* easiest via **Tools** → **Set** → **Create** → **Element**, then use **by angle** / **by feature** or box-select the hole rings. Keep this set as small as is useful — the recovery matrix cost scales with it.

Step 4 — Interaction module: kinematic coupling

1. **Module:** Interaction
2. **Constraint** → **Create** → **Coupling** → Continue.
3. **Control point:** pick **RP-1** (**RP_BOT**) → **Done**.
4. **Surface:** pick (**SURF_BOT**) → **Done**.
5. In the dialog: **Coupling type** = **Kinematic**. Check **U1 U2 U3 UR1 UR2 UR3** (all six). **OK**.
6. **Repeat** for **RP-2** / (**SURF_TOP**).

Why kinematic: rigid grips for a pull test (§14.1). If you later mate this to a compliant part, revisit §2.2.

Step 5 — Step module: substructure generation

1. **Module:** Step
2. **Model** → **Substructure** → **Create...** (if your CAE version lacks this, create a **Static, General** step and convert it via the **Keyword Editor** — see the note at the end)
3. Alternatively and more reliably: **Step** → **Create** → **Substructure Generation** → Continue.
4. In the dialog:
 - **Substructure identifier:** (**Z101**)
 - **Recovery matrix:** check "Whole model" or "Region" → choose **Region** → select (**ELS_HOLES**).
 - This is the GUI hook for stress recovery. If your version does not expose it, use the [KEYWORD EDITOR] block below.
 - **Mass matrix:** unchecked (static only).
5. **Retained degrees of freedom:** in the same dialog (or **Model** → **Substructure** → **Retained DOF**), add:
 - (**RP_BOT_SET**) → **DOF 1 through 6**
 - (**RP_TOP_SET**) → **DOF 1 through 6**
 - Check **Sorted**.
6. **OK**.

Step 6 — [KEYWORD EDITOR] Add recovery + matrix output

Model → **Edit Keywords** → <your model name>.

Find the (***SUBSTRUCTURE GENERATE**) block and make it read:

```
*SUBSTRUCTURE GENERATE, TYPE=Z101, OVERWRITE, RECOVERY MATRIX=YES, MASS  
MATRIX=NO  
*RETAINED NODAL DOFS, SORTED=YES  
RP_BOT_SET, 1, 6  
RP_TOP_SET, 1, 6  
*ELEMENT RECOVERY MATRIX, ELSET=ELS_HOLES  
S, E  
*SUBSTRUCTURE MATRIX OUTPUT, STIFFNESS=YES, FILE NAME=perf_K, OUTPUT  
FILE=USER DEFINED
```

These two lines are the ones with no GUI equivalent:

- `*ELEMENT RECOVERY MATRIX` — without it, **hole stress is unrecoverable** (§14.10).
- `*SUBSTRUCTURE MATRIX OUTPUT` — without it, **no readable** `.mtx` to parse.

OK to close the editor.

Step 7 — Job module: run generation

1. **Module: Job** → **Job** → **Create** → name `perf_gen` → Continue → **OK**.
2. **Job** → **Submit** → **perf_gen**.
3. **Job** → **Monitor** to watch for completion.
4. **Outputs:** `perf_gen.sim` (the superelement) and `perf_K.mtx` (the stiffness matrix — this is the file §14.9 parses).

JOB 2 (GUI) — Use the substructure (the pull test)

Step 1 — New model, import the superelement

1. **Model** → **Create** → name `PULL_TEST`.
2. **Model** → **Substructure** → **Import...**
3. Browse to the generated substructure library (`perf_gen`), select `Z101` → **OK**.
 - CAE creates a **substructure part** with the two retained reference nodes exposed.

Step 2 — Assembly

1. **Module: Assembly** → **Instance** → **Create** → select the imported substructure → **OK**.
2. The two retained RPs appear as selectable points. Create sets for them if CAE has not:
`RP_BOT_SET`, `RP_TOP_SET`.

Step 3 — Step module

1. **Step** → **Create** → **Static, General** → Continue.
2. Time period **1.0**; **Nlgeom: Off**. OK.
 - (A substructure is inherently linear — leave **NLGEOM** off.)

Step 4 — Load module: BC on the fixed end

1. **Module: Load**
2. **BC** → **Create** → Step: **Step-1** → **Mechanical** → **Displacement/Rotation** → Continue.
3. Select **RP_BOT** → **Done**.
4. Check **U1, U2, U3, UR1, UR2, UR3** — all = **0**. OK.
 - This grounds all six rigid-body modes.

Step 5 — Load module: the pull

Option A — prescribed displacement (recommended):

1. **BC** → **Create** → **Mechanical** → **Displacement/Rotation** → Continue.
2. Select **RP_TOP** → **Done**.
3. Set:
 - **U3 = 0.002985** (0.1% of $t = 2.985$ → clean linear pull)
 - **U1 = 0, U2 = 0** (no lateral drift)
 - **UR1 = UR2 = UR3 = 0** (rigid grip)
4. OK.
 - To *push*, use **$U3 = -0.002985$** .

Option B — applied force:

1. **Load** → **Create** → **Mechanical** → **Concentrated Force** → Continue.
2. Select **RP_TOP** → **Done**. **CF3 = 10000** (lb). OK.
3. Leave **U1/U2/UR** free (or restrain **U1, U2** to prevent drift).

Step 6 — Field/History output

1. **Output** → **Field Output Requests** → **Create** → Continue.
2. Check **U** (Translation/Rotation) and **RF** (Reaction forces).
3. Under **State/Field/User/Time**, also request **S** and **E** so recovered stress is written.
4. **Output** → **History Output Requests** → **Create** → Domain: **Set** → **RP_TOP_SET** → request **U3, RF3**.

Step 7 — [KEYWORD EDITOR] Invoke stress recovery

Model → Edit Keywords → PULL_TEST. Inside the `*STEP` block, add:

```
*SUBSTRUCTURE RECOVERY, ELSET=ESUB  
*ELEMENT OUTPUT, ELSET=ELS_HOLES  
S, E, MISES
```

Without `*SUBSTRUCTURE RECOVERY`, the recovery matrix from Job 1 exists but is **never invoked** — you will get interface reactions but **no hole stress**. This is a very easy step to skip.

Step 8 — Submit and read results

1. Job → Create → `perf_pull` → **Submit**.
2. **Module: Visualization** → open `perf_pull.odb`.

To read the reaction force:

- **Result → Field Output → Variable: RF, component RF3.**
- **Report → Field Output → Position: Unique Nodal → Variable: RF3 → select `RP_TOP_SET` → in the Setup tab check Sum → Apply.**
- The written report gives the total RF3 → `K_axial = RF3 / 0.002985`.
- (Or: **Tools → Query → Probe values → Nodes → click RP_TOP → read RF3.**)

To see the recovered hole stress:

- **Result → Field Output → Variable: S, Invariant: Mises.**
- Plot contours — you should see stress concentrations **at the hole edges**.
- **Sanity check (§14.8):** peak Mises at a hole edge should exceed the net-section average by roughly 2-3×. If the field is uniform or blank, the recovery matrix did not take — go back to Job 1 Step 6.

14.11.1 GUI gotchas

- **CAE substructure support is partial.** The two Keyword Editor blocks above are not optional workarounds — they are the only way to get recovery scoping and the readable `.mtx`.
- **Keyword edits are lost if you regenerate the model** from the GUI in some workflows. If a submit silently drops your recovery, re-open **Edit Keywords** and confirm the lines

are still there.

- **Independent instance** (not Dependent) in Job 1's assembly, or you cannot mesh/select on the instance cleanly.
 - **Reference points must exist in the Assembly**, not the Part, for coupling to the instance surfaces.
 - **ELS_HOLES** must be an element set, not a geometry set, for ***ELEMENT RECOVERY MATRIX**.
 - If **Model** → **Substructure** is missing from your CAE version, build the whole thing as a **Static** step and convert it entirely in **Edit Keywords** using the §14.2 block — the solver does not care that CAE never drew it.
-

14.12 CAE Keyword Corrections & Clarifications (from a real build)

Notes captured from an actual CAE-generated keyword file — the common failure points and how the lines should read.

14.12.1 Retained DOF — what they are and where they go

Retained DOF are the bridge between the substructure and the global assembly. The substructure eliminates *all* interior DOF and keeps only the retained ones — that is the entire mechanism. A few consequences people trip on:

- **A retained DOF is the opposite of a boundary condition.** A BC *removes* a DOF; a retained DOF *preserves* it through condensation. So retained DOF are **not** set in the Load module and **not** entered as a **Displacement/Rotation** BC.
- They belong to **generation** (***Substructure Generate**), not loading.
- CAE frequently does not expose the retained-DOF dialog. If you do not see it, add the line directly in **Edit Keywords**, immediately under ***Substructure Generate**:

```
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
```

If this line is missing, the substructure retains nothing and the job errors or produces a useless superelement. This is the most common first-run failure.

14.12.2 Recovery scoping — two valid forms (do not use both)

To keep interior stress around the holes, recovery can be requested **two** ways:

Form 1 — shorthand **elset=** on the generate line (what CAE writes):

```
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,  
elset=ELS_HOLES  
*Retained Nodal Dofs, sorted=YES  
RP_BOT_SET, 1, 6  
RP_TOP_SET, 1, 6
```

Here `recovery matrix=YES` turns recovery on, and `elset=ELS_HOLES` scopes it to the hole-ring elements. **No separate `*Element Recovery Matrix` line is needed** — this form is self-contained.

Form 2 — long form with an explicit output selection:

```
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES  
*Retained Nodal Dofs, sorted=YES  
RP_BOT_SET, 1, 6  
RP_TOP_SET, 1, 6  
*Element Recovery Matrix, elset=ELS_HOLES  
S, E
```

Use one or the other, not both. Form 1 is simplest and is what CAE generates; reach for Form 2 only if you want to name specific output variables in the recovery matrix. (An earlier draft of §14.2 over-specified by combining them — Form 1 alone is correct.)

`ELS_HOLES` **must be an `*Elset`** (element set), not an `*Nset`. `*Element Recovery Matrix` and the `elset=` shorthand both require elements.

14.12.3 `*Substructure Matrix Output` — where it goes

It lives **inside the step**, after the generate/retained/recovery lines:

```
*Step, name=Step-1, nlgeom=NO  
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,  
elset=ELS_HOLES  
*Retained Nodal Dofs, sorted=YES  
RP_BOT_SET, 1, 6  
RP_TOP_SET, 1, 6  
*Substructure Matrix Output, stiffness=YES, file name=perf_K, output  
file=USER_DEFINED  
*End Step
```

This writes the readable `perf_K.mtx` (§14.9). Without it there is no ASCII stiffness matrix to parse.

14.12.4 Reference-node naming consistency

CAE may auto-name a reference point's set (e.g. `m_Set-4`) instead of the name you intended. Whatever the bottom RP set is actually called, **the coupling, retained-DOF, BC, and output lines must all use the same name**. Cleanest fix: rename it to `RP_BOT_SET` in CAE (Assembly → Set Manager → Rename) so every reference is consistent. Corrected coupling line:

```
*Coupling, constraint name=kin_coupling_bot, ref node=RP_BOT_SET,  
surface=SURF_BOT  
*Kinematic
```

14.12.5 Delete `*Damping Controls`

CAE may insert `*Damping Controls, structural=COMBINED, viscous=COMBINED` into the generation step. For a **static, stiffness-only** generation it is unnecessary and implies dynamic intent you do not have. Remove it.

14.12.6 The generation job produces (almost) no ODB — this is expected

A substructure **generation** job is a reduction, not an analysis. Nothing is solved, so there are no results to view. Do not expect stress, displacement, or a responsive model in the generation ODB — it will be empty or trivial, and that is **correct, not a failure**.

Generation writes instead:

- `Z101.sim` / `.sup` — the superelement (the real deliverable)
- `perf_K.mtx` — the stiffness matrix
- `.prt`, `.stt`, and library `.mtx` files

Interior stress only appears in Job 2, after the superelement is loaded and `*Substructure Recovery` is invoked (§14.7). If you submit generation and open the ODB expecting to see the tubesheet respond, you will think it failed — it did not; the superelement lives in the `.sim` / `.sup` library, not the ODB.

14.12.7 Corrected minimal generation step (all fixes applied)

```
** MATERIALS  
*Material, name=cf_650  
*Elastic  
2.7e+07, 0.3  
**  
** (ELS_HOLES must be an *Elset of the hole-ring elements)  
** (RP_BOT_SET, RP_TOP_SET are the two reference-node sets)
```

```

**
** STEP
*Step, name=Step-1, nlgeom=NO
*Substructure Generate, overwrite, type=Z101, recovery matrix=YES,
elset=ELS_HOLES
*Retained Nodal Dofs, sorted=YES
RP_BOT_SET, 1, 6
RP_TOP_SET, 1, 6
*Substructure Matrix Output, stiffness=YES, file name=perf_K, output
file=USER_DEFINED
*End Step

```

14.12.8 Job 2 — also read the bottom (fixed-end) reaction

`RP_BOT` is fully fixed, so its reaction is the grounded reaction — read it alongside the top. It should be **equal and opposite** to the top reaction (a built-in equilibrium check).

Add both reference sets to the output, and report both:

```

*OUTPUT, HISTORY
*NODE OUTPUT, NSET=RP_TOP_SET
U3, RF3
*NODE OUTPUT, NSET=RP_BOT_SET
RF3

```

In Visualization → **Report** → **Field Output** → Variable **RF3**, Position **Unique Nodal**, select `RP_BOT_SET`, check **Sum** → the summed `RF3` at the bottom should equal `-RF3(top)` to within solver noise. If it does not, an interior constraint or the coupling is leaking load — investigate before trusting the stiffness.

14.13 Reading the Stiffness Matrix — Layout, Math, and a Parser

14.13.1 The `.mtx` is lower-triangular — how to see rows and columns

Abaqus writes only the **lower triangle** of the symmetric matrix, one matrix **row per block**, wrapping long rows across text lines. So "row i " contains exactly i numbers: `K(i,1), K(i,2), ..., K(i,i)`. The upper triangle is implied by symmetry (`K(j,i) = K(i,j)`).

DOF index → **(node, direction) map** for a 2-node superelement:

```

index :   1   2   3   4   5   6 |   7   8   9  10  11  12
node  :  ----- node 1 ----- | ----- node 2 -----
dof   :  u1  u2  u3  r1  r2  r3 |  u1  u2  u3  r1  r2  r3
          (X) (Y) (Z) (RX) (RY) (RZ) | (X) (Y) (Z) (RX) (RY) (RZ)

```

Full 12×12 assembled from your 3-hole matrix (symmetric; diagonals bold, $\boxed{\cdot}$ = numerical-zero noise). Values rounded:

	1	2	3	4	5	6	7	8
9	10	11	12					
	(n1 X)	(n1 Y)	(n1 Z)	(n1 RX)	(n1 RY)	(n1 RZ)	(n2 X)	(n2 Y)
	(n2 Z)	(n2 RX)	(n2 RY)	(n2 RZ)				
1	[2.355e7	163	.	.	.	-1.23e5	-2.355e7	-163
.	.	.	1.24e5]					
2	[163	5.397e6	.	.	.	5.396e6	-163	
-5.397e6	.	.	.	5.398e6]				
3	[.	.	3.770e6	1.19e4	-3.770e6	.	.	.
-3.770e6	-1.19e4	3.770e6	.	]				
4	[.	.	1.19e4	2.616e6	-1.17e4	.	.	.
-1.19e4	-2.616e6	1.21e4	.	]				
5	[.	.	-3.770e6	-1.17e4	5.772e6	.	.	.
3.770e6	1.17e4	-3.771e6	.	]				
6	[-1.23e5	5.396e6	.	.	.	1.439e7	1.23e5	
-5.396e6	.	.	.	-3.597e6]				
7	[-2.355e7	-163	.	.	.	1.23e5	2.355e7	163
.	.	.	-1.24e5]					
8	[-163	-5.397e6	.	.	.	-5.396e6	163	5.397e6
.	.	.	-5.398e6]					
9	[.	.	-3.770e6	-1.19e4	3.770e6	.	.	.
3.770e6	1.19e4	-3.771e6	.	]				
10	[.	.	-1.19e4	-2.616e6	1.17e4	.	.	.
1.19e4	2.616e6	-1.21e4	.	]				
11	[.	.	3.770e6	1.21e4	-3.771e6	.	.	.
-3.771e6	-1.21e4	5.773e6	.	]				
12	[1.24e5	5.398e6	.	.	.	-3.597e6	-1.24e5	
-5.398e6	.	.	.	1.439e7]				

What to look for, in order:

1. **The diagonal** (top-left to bottom-right): the 12 direct stiffnesses. $\boxed{K(1,1)=2.355e7}$ is DOF-1 (node-1 X); $\boxed{K(7,7)=2.355e7}$ is DOF-7 (node-2 X). **They match → symmetric part. ✓**

2. The four 6×6 quadrants:

- **Top-left (rows 1–6, cols 1–6):** node 1's self-stiffness.
- **Bottom-right (rows 7–12, cols 7–12):** node 2's self-stiffness — **should mirror top-left.** ✓
- **Off-diagonal quadrants (rows 7–12 × cols 1–6):** node1↔node2 coupling. For each translation you see $-K(\text{diag})$ (e.g. $K(7,1) = -2.355e7$) → the $\begin{bmatrix} k, & -k \\ -k, & k \end{bmatrix}$ spring pattern.

3. **Within a node block:** big off-diagonals like $K(6,2)=5.4e6$ and $K(5,3)=-3.77e6$ are **real lever-arm couplings** (rotation ↔ translation from the rigid coupling). The $\begin{pmatrix} \cdot \end{pmatrix}$ entries ($\sim 1e-4$) are numerical zeros — ignore.

14.13.2 The math — how stiffness comes out of the matrix

The reduced element obeys $\{f\} = K \cdot \{u\}$, where $\{u\}$ and $\{f\}$ are the 12 retained DOF displacements and forces:

$$\begin{bmatrix} f_1 \\ \vdots \\ f_{12} \end{bmatrix} = \begin{bmatrix} K(1,1) & \dots & K(1,12) \\ \vdots & \ddots & \vdots \\ K(12,1) & \dots & K(12,12) \end{bmatrix} \begin{bmatrix} u_1 \\ \vdots \\ u_{12} \end{bmatrix}$$

Your test: $\{RP_BOT\}$ fully fixed ($\{u_1 \dots u_6\} = 0$), $\{RP_TOP\}$ given $\{u_7 = U_1 = 0.002\}$ (X), all other $\{RP_TOP\}$ DOF fixed at 0. So the only nonzero displacement is $\{u_7\}$. The force needed at DOF 7 is:

$$f_7 = K(7,7) \cdot u_7 = 2.355e7 \times 0.002 = 4.71e4 \text{ lb} \quad \leftarrow \text{this is RF1 at RP_TOP}$$

and equilibrium gives the fixed-end reaction:

$$f_1 = K(1,7) \cdot u_7 = -2.355e7 \times 0.002 = -4.71e4 \text{ lb} \quad \leftarrow \text{RF1 at RP_BOT}$$

Therefore the directional stiffness is just:

$$K_{dir} = RF_{dir} / U_{dir} = 4.71e4 / 0.002 = 2.355e7 \text{ lb/in}$$

which equals $K(7,7)$ (and $K(1,1)$) — the diagonal for the pulled DOF. **Your measured RF = 4.71e4 at U = 2e-3 confirms this exactly.** ✓

General rule — the stiffness you extract is the diagonal of the DOF you pulled:

- Pull in +X (DOF 1 / 7) → you get $K(1,1)$ = X-extension stiffness.
- Pull in +Y (DOF 2 / 8) → $K(2,2)$.

- Pull in +Z (DOF 3 / 9) → $K(3,3)$.
- Apply a rotation (DOF 4/5/6) → the corresponding rotational stiffness.

This is why the pull direction matters: **each axis reads a different diagonal**. To characterize all directions, pull along each axis in turn (or read the diagonals straight from the $.mtx$).

Caveat — off-axis coupling. $K_{dir} = RF/U$ gives the *pure* directional stiffness **only** because you fixed all other DOF at RP_TOP. If you had left the other DOF free, the node would also translate/rotate in coupled directions (via terms like $K(6,2)$), and RF/U would blend several terms. Fixing the non-loaded DOF (as your BC-2 does) is what isolates the single diagonal. Keep doing that for clean stiffness reads.

14.13.3 Python parser (standard library only)

Reads the $.mtx$, rebuilds the full symmetric matrix, prints it neatly, and reports the directional stiffness for whatever axis you pulled.

```
python
```

```

#!/usr/bin/env python3
"""
read_substructure_stiffness.py
Parse an Abaqus *SUBSTRUCTURE MATRIX OUTPUT .mtx (lower-triangular),
rebuild the full symmetric stiffness, and report directional stiffness.
Standard library only.
"""

# ----- INPUTS -----
MTX_FILE = "perf_K.mtx"      # Abaqus matrix file
N_NODES  = 2                 # retained reference nodes
NDOF     = 6                 # DOF per node (3 trans + 3 rot)

# For a stiffness readout, tell it what you pulled:
PULL_NODE_INDEX = 2          # 1 = first retained node, 2 = second, ...
PULL_DOF        = 1          # 1=X 2=Y 3=Z 4=RX 5=RY 6=RZ
APPLIED_U       = 0.002      # prescribed displacement you applied
MEASURED_RF     = 4.71e4      # reaction force you read (optional cross-check; No
# -----

N = N_NODES * NDOF
DOF_NAMES = ["X", "Y", "Z", "RX", "RY", "RZ"]

def parse_mtx(path):
    """Return full NxN symmetric matrix from a lower-triangular .mtx."""
    vals = []
    with open(path) as f:
        for line in f:
            s = line.strip()
            if not s or s.startswith("#") or s.startswith("***):
                continue
            # numbers may be comma- and/or space-separated; Fortran 'E' exponent
            for tok in s.replace(",", " ").split():
                try:
                    vals.append(float(tok))
                except ValueError:
                    pass # skip stray labels

    K = [[0.0] * N for _ in range(N)]
    k = 0
    # lower-triangular fill: row i has (i+1) entries
    for i in range(N):

```

```

        for j in range(i + 1):
            if k >= len(vals):
                raise ValueError(
                    "Ran out of numbers: expected %d lower-tri entries, got %d. "
                    "Check N_NODES/NDOF." % (N * (N + 1) // 2, len(vals)))
            K[i][j] = vals[k]
            K[j][i] = vals[k]          # mirror to upper triangle
            k += 1
    return K

def fmt(x):
    """Compact scientific / dotted-zero formatting for display."""
    if abs(x) < 1e-3 * 1.0:          # relative noise threshold applied later
        return x
    return x

def print_matrix(K):
    # column headers
    hdr = []
    for a in range(N_NODES):
        for d in range(NDOF):
            hdr.append("n%d%s" % (a + 1, DOF_NAMES[d]))
    # noise threshold = 1e-6 of the largest magnitude
    mx = max(abs(K[i][j]) for i in range(N) for j in range(N))
    tol = 1e-6 * mx

    colw = 12
    print(" " * 6 + "".join("%*s" % (colw, h) for h in hdr))
    for i in range(N):
        row = "%4s " % hdr[i]
        for j in range(N):
            v = K[i][j]
            if abs(v) < tol:
                cell = "."
            else:
                cell = "%.4g" % v
            row += "%*s" % (colw, cell)
        print(row)

def check_health(K):
    # symmetry

```

```

mx = max(abs(K[i][j]) for i in range(N) for j in range(N))
asym = max(abs(K[i][j] - K[j][i]) for i in range(N) for j in range(N)) / mx
print("\nSymmetry error (max|K-K^T|/max|K|) = %.2e (want ~0)" % asym)

# node-block diagonal twins (only meaningful for a symmetric 2-node part)
if N_NODES == 2:
    print("Diagonal twins (node1 vs node2):")
    for d in range(NDOF):
        a, b = K[d][d], K[NDOF + d][NDOF + d]
        rel = abs(a - b) / max(abs(a), 1e-30)
        flag = "ok" if rel < 1e-3 else "DIFFER"
        print(" %-3s : %-12.5g %-12.5g  %s"
              % (DOF_NAMES[d], a, b, flag))

def directional_stiffness(K):
    idx = (PULL_NODE_INDEX - 1) * NDOF + (PULL_DOF - 1)
    kdiag = K[idx][idx]
    name = "node%d %s" % (PULL_NODE_INDEX, DOF_NAMES[PULL_DOF - 1])
    print("\n--- Directional stiffness ---")
    print("Pulled DOF: %s (global index %d)" % (name, idx + 1))
    print("K(%d,%d) from matrix          = %.6g lb/in" % (idx + 1, idx + 1, kdiag))
    if MEASURED_RF is not None and APPLIED_U:
        kmeas = MEASURED_RF / APPLIED_U
        rel = abs(kmeas - kdiag) / max(abs(kdiag), 1e-30)
        print("K = RF/U = %.4g / %.4g = %.6g lb/in" % (MEASURED_RF, APPLIED_U,
        print("Agreement                    = %.3f%% (want < 1%%)" % (100 * rel))
    return kdiag

if __name__ == "__main__":
    K = parse_mtx(MTX_FILE)
    print("Full %dx%d reduced stiffness matrix (. = |value| < 1e-6*max):\n" % (
    print_matrix(K)
    check_health(K)
    directional_stiffness(K)

```

To reuse for a different pull direction: change `PULL_DOF` (1=X, 2=Y, 3=Z, 4/5/6 = rotations) and `PULL_NODE_INDEX`, and paste in that run's `APPLIED_U` / `MEASURED_RF`. The script reports both the matrix diagonal for that DOF and the `RF/U` cross-check. It also scales to more nodes/DOF by changing `N_NODES` and `NDOF` — the lower-triangular fill is general.

14.14 Validating the §10 Hand-Calc Constants with Directional Pull Tests

Goal: use the substructure directional stiffness (§14.13) to check the equivalent-solid moduli from §10 ($E_1 = 0.683 \cdot E_m$, $E_2 = E_3 = 0.34 \cdot E_m$).

14.14.1 Yes — three pulls, one per axis

Run the §14 pull test three times, pulling along X, then Y, then Z, each time fixing the opposite RP and all non-loaded DOF at the pulled RP (§14.13.2 — this is what isolates a single diagonal). Each run yields one directional stiffness:

- Pull **Z** → (K_z) → verifies E_1 (through-thickness)
- Pull **Y** → (K_y) → verifies E_2 (in-plane weak, across B-C)
- Pull **X** → (K_x) → verifies E_3 (in-plane strong)

14.14.2 The bridge: structural stiffness → material modulus

The substructure gives a **structural** stiffness (K) (lb/in). The §10 card is a **material** modulus (E) (psi). Convert:

$$E_{\text{dir}} = K_{\text{dir}} \cdot L / A_{\text{gross}}$$

- (L) = the pull span (distance between the two RP faces along the pull axis)
- (A_{gross}) = the **gross** cross-section perpendicular to the pull (**not** net — the equivalent-solid model has no holes, so it works in gross area, and that is what you are validating against)

Then compare:

$$\begin{array}{llll} E_z / E_m & \text{vs} & 0.683 & (\text{through-thickness}) \\ E_y / E_m & \text{vs} & 0.340 & (\text{in-plane, weak}) \\ E_x / E_m & \text{vs} & 0.340 & (\text{in-plane, strong}) \end{array}$$

14.14.3 CRITICAL — use a rectangular block, not the hexagon

For $(E_{\text{dir}} = K \cdot L / A)$ to yield a clean modulus, the specimen must be a **prismatic rectangular block cut from the tube field** — flat parallel faces, well-defined (L) and (A) . **Do not run this on the hexagon.** As established in §13.B.0, a hexagon's angled faces produce a non-uniaxial stress state, an ill-defined $(L)/(A)$, and reference-node reactions that blend directions — the validation number would be contaminated.

So: build the rectangular block (integer number of lattice cells, holes meshed, isotropic base metal, RPs on two opposite faces), generate its substructure, and run the three pulls. This is

Stage A/B of §13 expressed as substructure pulls rather than direct FE — same specimen, same logic.

14.14.4 Interpreting the comparison

- **E_z vs 0.683·E_m:** expect a close match (~1–3%). This is a near-exact closed form; a large gap means a mesh/BC/area error.
- **E_y vs 0.34·E_m (weak):** the card was built at this direction's η , so it should match well.
- **E_x vs 0.34·E_m (strong):** expect the substructure to come out **stiffer** than 0.34·E_m — because the real array is stiffer in the strong direction than the worst-case isotropic card admits. That gap is the **error bar on the isotropic simplification** (§10.5), quantified. (E_x/E_y) from the pulls is the true in-plane orthotropy ratio.

Use the §14.13.3 Python parser three times (change `PULL_DOF` to 1/2/3), or read `K(1,1)`, `K(2,2)`, `K(3,3)` straight off a single `.mtx` — the three diagonals give all three directions from one generation.

14.15 How Substructure Generation Works — Q&A

14.15.1 How did Abaqus build the matrix with no load applied?

A load was never needed — stiffness is a property of geometry and material, not a response to load.

Every finite element has a stiffness matrix computed purely from its shape functions, material, and coordinates:

$$k_e = \int B^T D B \, dV \quad (B = \text{strain-displacement, } D = \text{material stiffness})$$

No load appears in that integral. Abaqus:

1. Assembles the **full interior stiffness** `K` of the meshed part from all `ke`.
2. Partitions DOF into **retained** (`r`, your RPs) and **internal** (`i`, everything else).
3. **Statically condenses** the internal DOF:

$$K_{red} = K_{rr} - K_{ri} K_{ii}^{-1} K_{ir}$$

4. Writes `Kred` to the `.mtx`.

The reference points and their kinematic coupling define *which* DOF are retained; the coupling ties each face's motion to its RP. **Generation is "measuring the part's spring constant," which is a property — no load required.** Loads enter only in the usage job.

14.15.2 Must the RP be at the centroid, and must it be a node?

Centroid — recommended, not required.

- With **kinematic** coupling the face is rigid, so the RP *location* does **not** change the reduced stiffness — any point rigidly tied to the face gives an equivalent spring.
- What it *does* change is the **reference frame for the moment/rotation terms**. An off-centroid RP mixes force and moment in the coupling block (a lever arm), making the matrix harder to read and hand-check. The centroid keeps translation and rotation DOF cleanly separated.
- **An offset RP is valid** — just less tidy to interpret. If you have a reason to place it elsewhere (e.g. it must align with a mating part's connection point), that is fine.

Must it be a node — yes. A reference point *is* a node: a single node not attached to any element, carrying the retained DOF and connected to the structure through the coupling. It does not need to lie on existing geometry, but it is itself a node in the model.

14.15.3 Skipping the recovery matrix — file size and trade-offs

Yes, skipping recovery meaningfully shrinks the files and speeds up the job. The recovery matrix stores the map from retained-DOF motion back to the response of **every recovered interior element** — typically the largest data structure a substructure produces.

	Recovery ON (scoped to ELS_HOLES)	Recovery OFF
File size	larger (grows with recovered element count)	smallest
Generation time	slower	faster
Usage time	slower (recovery replayed)	faster
Interior stress/strain	recoverable in ELS_HOLES	permanently unavailable
Add later?	—	No — must regenerate

Rules:

- Need interior stress → request recovery, **scoped tightly** (only the elements whose stress matters, e.g. hole rings). Cost scales with the recovered set, so do not pass the whole part.

- Need the part only as a **stiffness spring** (most global-model uses) → **skip recovery entirely** for the smallest, fastest superelement.

14.15.4 Can I load a substructure, and only at the RPs?

Yes, and effectively yes — loads/BCs go at the retained DOF. After condensation, the retained reference-node DOF are the **only** DOF the reduced element exposes to the outside world; everything interior was eliminated. So in the usage job you apply forces, displacements, and BCs at the retained RPs.

- **Exception:** Abaqus supports **substructure loads** — preset internal load patterns (e.g. a gravity or pressure case) baked in **at generation time** and later scaled in usage. But these must be defined *during* generation; you cannot introduce new interior loads after the fact. For interface-driven mechanical loading, the retained RPs are where everything happens.

14.15.5 What if I retain multiple reference points (top, bottom, left face)?

Three RPs × 6 DOF = an 18×18 reduced matrix, structured as a 3×3 grid of 6×6 blocks:

	RP_top	RP_bot	RP_left	
RP_top	K _{tt}	K _{tb}	K _{tl}	diagonal blocks = each interface's SELF-stiffness off-diag blocks = CROSS-coupling (load one interface → reaction at another)
RP_bot	K _{bt}	K _{bb}	K _{bl}	
RP_left	K _{lt}	K _{lb}	K _{ll}	

What to infer:

- **Diagonal 6×6 blocks** (K_{tt} , K_{bb} , K_{ll}): the direct stiffness of each interface — pull that interface, get its own reaction.
- **Off-diagonal 6×6 blocks** (K_{tl} , etc.): how motion at one interface produces reaction at another — the load-transfer paths through the part. K_{tl} = the reaction at the left face when the top is displaced.
- **Symmetry:** $K_{tl} = K_{lt}^T$ (reciprocity).
- **Free-free rank:** still exactly **6 rigid-body modes**, regardless of interface count.
- It is no longer a two-terminal spring — it is a **multi-port** stiffness element. This is exactly what your real tubesheet and spool superelements are: reduced elements with several interfaces, each a 6-DOF port, fully coupled through the condensed interior.

Reading it follows the same rules as the 12×12 (§14.13): diagonals are direct stiffnesses, off-diagonal blocks are couplings, near-zero terms are numerical noise. The Python parser (§14.13.3) handles it directly — set `N_NODES = 3`.